

Lab 1: Software Testing Fundamentals

1. Why is Testing Necessary? (True/False)

Based on the provided points, here is the validation of common reasons for testing:

- **True:** Because software is likely to have faults.
- **True:** To learn about the reliability of the software.
- **False:** To fill the time between the delivery of the software and the release date.
- **False:** To prove that the software has no faults.
- **True:** Because testing is included in the project plan.
- **True:** Because failures can be very expensive.
- **True:** To avoid being sued by customers.
- **True:** To stay in business.

2. Comparisons

Functional vs. Non-Functional Testing

- **Functional Testing:** Focuses on **what** the system does (requirements and specifications).
- **Non-Functional Testing:** Focuses on **how** the system works (performance, usability, and reliability).

Black Box, White Box, and Gray Box Testing

- **Black Box:** Testing without knowledge of the internal code or structure.
- **White Box:** Testing with full access to the internal code and program logic.
- **Gray Box:** Testing with partial knowledge of the internal structure (e.g., database access).

3. What is the SDLC?

The **Software Development Life Cycle (SDLC)** is a process used by the software industry to design, develop, and test high-quality software. It consists of a detailed plan describing how to develop, maintain, replace, and alter or enhance specific software.

4. What are the Testing Levels?

1. **Unit Testing:** Testing individual components.
2. **Integration Testing:** Testing the interactions between components.
3. **System Testing:** Testing the complete, integrated system.
4. **Acceptance Testing:** Final testing before delivery to the end user.

5. What are the Testing Types?

Common types include:

- **Regression Testing:** Ensuring changes haven't introduced new bugs.
- **Smoke Testing:** Checking basic functionality.
- **Performance Testing:** Checking speed and stability.
- **Security Testing:** Identifying vulnerabilities.

6. [Bonus] TDD vs. DDD vs. BDD

- **TDD (Test-Driven Development):** Writing tests before writing the actual code.
- **BDD (Behavior-Driven Development):** Focusing on the behavior of the application for the user (Given/When/Then).
- **DDD (Domain-Driven Design):** Designing software based on the business domain and logic.

7. Who does Testing?

- **Software Testers / QA Engineers**
- **Developers** (Unit and Integration)
- **Project Managers**
- **End Users** (User Acceptance Testing)